



in collaboration with



Title:

Lab 2 Task

Course:

ST5003CMD-Operating Systems, Security and Network

Institution:

Softwarica College of IT and E-Commerce

Submitted By:

Sandip Rawal (240684)

Submitted To:

Mr.Rupak Rajbanshi

Submission Date:

September 20, 2026

Table of Contents:

I. Introduction.....	4
II. C Program Structure and Examples.....	5
A. Basic Hello World Program.....	5
B. Program with User Input.....	5
C. Program with Formatted Output.....	6
III. GCC Compilation Process.....	7
A. Installing GCC on Linux.....	7
B. Four Stages of C Compilation.....	7
C. Compilation Stage Examples.....	8
1. Stage 1: Preprocessing:.....	8
2. Stage 2: Compilation to Assembly.....	9
3. Stage 3: Assembly to Object File.....	9
4. Stage 4: Linking.....	9
IV. Return Values and Exit Status.....	10
A. Understanding Return Values.....	10
B. Checking Exit Status.....	10
V. Knowledge Check Questions.....	11
VI. Conclusion.....	12

List of Figures:

Figure 1: Hello World.....	5
Figure 2: User Input.....	5
Figure 3: Formatted Output.....	6
Figure 5:GCC Install.....	7
Figure 6: Table showing Four Stages.....	7
Figure 7: Preprocessing(i file).....	8
Figure 8: Compilation.....	9
Figure 9: Assembly.....	9
Figure 10: Linking.....	9
Figure 11: Return Value.....	10
Figure 12: Exit Status.....	10
Figure 13: C Program.....	11

I. Introduction

This lab report documents the practical work completed for **Lab 2: C Programming Basics** as part of the Operating Systems, Security and Networks module. The purpose of this lab is to develop a basic understanding of C programming and the process of compiling and executing C programs in a Linux environment. The practical activities were carried out using **Ubuntu Linux and the GCC (GNU Compiler Collection)**.

The lab covers the basic structure of a C program, including the use of header files, the `main()` function, variables, input/output functions, and return values. Several simple C programs were created to demonstrate **Hello World output, user input, and formatted output**. The lab also explores the four main stages of the GCC compilation process: **preprocessing, compilation, assembly, and linking**, including the different file types generated at each stage.

In addition, the lab demonstrates how program **return values and exit statuses** work in Linux using the `echo $?` command. The practical tasks provide experience with creating, compiling, executing, and testing C programs through the Linux terminal. The completed source code, compilation outputs, screenshots, and knowledge check answers are documented as evidence of the practical work.

II. C Program Structure and Examples

A. Basic Hello World Program

The program is compiled with `gcc` and run, printing "Hello, World!" to the terminal.



The figure consists of two screenshots. The top screenshot is a terminal window showing the compilation and execution of a C program. The commands and output are as follows:

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ nano hello.c
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ gcc hello.c -o hello
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ ./hello
Hello, World!
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$
```

The bottom screenshot shows the source code of the program in the GNU nano 8.7.1 editor:

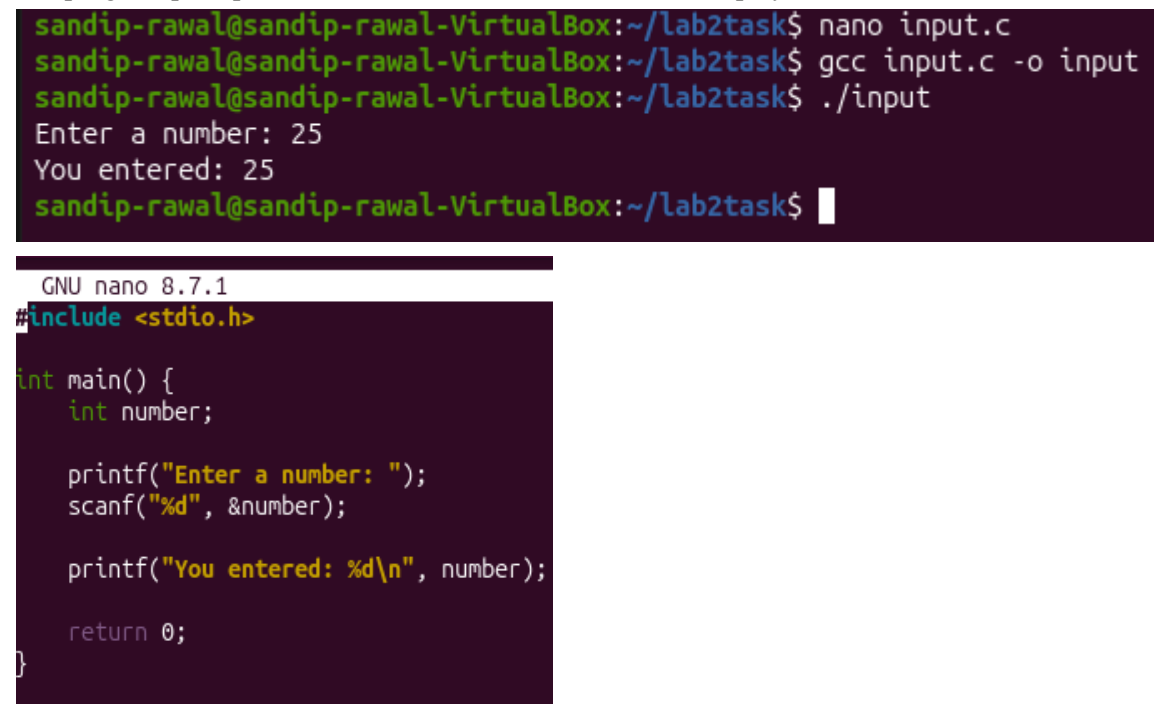
```
GNU nano 8.7.1
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Figure 1: Hello World

B. Program with User Input

The program prompts for a number, reads it via `scanf`, and displays it back to the user.



The figure consists of two screenshots. The top screenshot is a terminal window showing the compilation and execution of a C program. The commands and output are as follows:

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ nano input.c
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ gcc input.c -o input
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ ./input
Enter a number: 25
You entered: 25
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$
```

The bottom screenshot shows the source code of the program in the GNU nano 8.7.1 editor:

```
GNU nano 8.7.1
#include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);

    return 0;
}
```

Figure 2: User Input

C. Program with Formatted Output

User inputs (name, age, height) are read and displayed in a formatted output block.

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ nano formatted.c
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ gcc formatted.c -o formatted
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ ./formatted
Enter your name: sandip
Enter your age: 20
Enter your height (m): 60

--- User Info ---
Name: sandip
Age: 20 years
Height: 60.00 m
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$
```

```
GNU nano 8.7.1
#include <stdio.h>

int main() {
    char name[50];
    int age;
    float height;

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Enter your age: ");
    scanf("%d", &age);

    printf("Enter your height (m): ");
    scanf("%f", &height);

    printf("\n--- User Info ---\n");
    printf("Name: %s\n", name);
    printf("Age: %d years\n", age);
    printf("Height: %.2f m\n", height);

    return 0;
}
```

Figure 3: Formatted Output

III. GCC Compilation Process

A. Installing GCC on Linux

GCC is installed using `apt`, and the installed version is verified.

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ sudo apt update
Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Hit:2 http://np.archive.ubuntu.com/ubuntu resolute InRelease
Get:3 http://np.archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://np.archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]

sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ sudo apt install build-essential
Installing:
  build-essential

Installing dependencies:
  g++ g++-15 g++-15-x86-64-linux-gnu g++-x86-64-linux-gnu libcrypt-dev libstdc++-15-dev

sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Figure 5:GCC Install

B. Four Stages of C Compilation

Stage	Command	Output	Purpose
1. Preprocessing	<code>gcc -E hello.c -o hello.i</code>	<code>.i</code> file	Expand headers & macros
2. Compiling	<code>gcc -S hello.i -o hello.s</code>	<code>.s</code> file	Generate Assembly code
3. Assembling	<code>gcc -c hello.s -o hello.o</code>	<code>.o</code> file	Create object file
4. Linking	<code>gcc hello.o -o hello</code>	executable	Final executable

Figure 6: Table showing Four Stages

The compilation of a C program using GCC takes place in four sequential stages, each transforming the source code into a form closer to an executable program. In the **preprocessing** stage, the command `gcc -E hello.c -o hello.i` expands header files and macros within the source code, producing an intermediate file named `hello.i`. Next, in the **compiling** stage, the command `gcc -S hello.i -o hello.s` translates this preprocessed code into assembly language, generating the file `hello.s`. This is followed by the **assembling** stage, where the command `gcc -c hello.s -o hello.o` converts the assembly code into machine-readable object code, producing the object file `hello.o`. Finally, in the **linking** stage, the command `gcc hello.o -o hello` combines the object file with any required libraries to produce the final executable, `hello`. Together, these four stages illustrate how a single line of source code is systematically transformed into a program that can be executed directly by the operating system.

C. Compilation Stage Examples

1. Stage 1: Preprocessing:

The preprocessing stage expands header files and macros, producing the `.i` file.

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ gcc -E hello.c -o hello.i
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ cat hello.i | less
```

```
# 0 "hello.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "hello.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 431 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4
# 432 "/usr/include/features.h" 2 3 4
# 539 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 730 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 731 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 732 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 540 "/usr/include/features.h" 2 3 4
# 563 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 1 3 4
# 10 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs-64.h" 1 3 4
# 11 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 2 3 4
# 564 "/usr/include/features.h" 2 3 4
# 34 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 2 3 4
# 29 "/usr/include/stdio.h" 2 3 4
.
```

Figure 7: Preprocessing(i file)

2. Stage 2: Compilation to Assembly

The compilation stage translates the C code into assembly, producing the `.s` file.

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ gcc -S hello.c -o hello.s
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ cat hello.s
.file "hello.c"
.text
.section .rodata
.LC0:
.string "Hello, World!"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
endbr64
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
leaq .LC0(%rip), %rax
movq %rax, %rdi
call puts@PLT
movl $0, %eax
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE0:
.size main, .-main
.ident "GCC: (Ubuntu 15.2.0-16ubuntu1) 15.2.0"
.section .note.GNU-stack,"",@progbits
.section .note.gnu.property,"a"
.align 8
.long 1f - 0f
.long 4f - 1f
.long 5
0:
.string "GNU"
1:
.align 8
.long 0xc0000002
.long 3f - 2f
2:
.long 0x3
3:
.align 8
4:
```

Figure 8: Compilation

3. Stage 3: Assembly to Object File

The assembly stage converts assembly code into machine-level object code (`.o` file).

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ gcc -c hello.c -o hello.o
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ file hello.o
hello.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$
```

Figure 9: Assembly

4. Stage 4: Linking

The linking stage combines the object file with required libraries to produce the final executable.

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ gcc hello.o -o hello
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ ./hello
Hello, World!
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$
```

Figure 10: Linking

IV. Return Values and Exit Status

A. Understanding Return Values

Running the program and checking `echo $?` confirms a return value of 0, indicating success.

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ nano return.c
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ gcc return.c -o return
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ ./return
Program executed successfully
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ echo $?
0
```

```
GNU nano 8.7.1
#include <stdio.h>

int main() {
    printf("Program executed successfully\n");
    return 0;
}
```

Figure 11: Return Value

B. Checking Exit Status

After changing the return value to 1, `echo $?` confirms the non-zero exit status

```
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ nano return.c
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ gcc return.c -o return
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ ./return
Program executed with an error
sandip-rawal@sandip-rawal-VirtualBox:~/lab2task$ echo $?
1
```

```
GNU nano 8.7.1
#include <stdio.h>

int main() {
    printf("Program executed with an error\n");
    return 1;
}
```

Figure 12: Exit Status

V. Knowledge Check Questions

Question 1

What is the purpose of the `#include` directive in C programming? Provide an example of its usage.

-It includes header files in a C program; for example, `#include <stdio.h>` provides functions such as `printf()` and `scanf()`.

Question 2

Explain the difference between the four stages of compilation. What file extension is produced at each stage?

-Preprocessing produces `.i`, compilation produces `.s`, assembly produces `.o`, and linking produces the final executable.

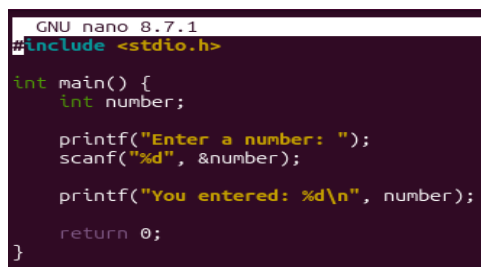
Question 3

What does a return value of 0 from the `main()` function indicate to the operating system? What about a non-zero return value?

-0 normally indicates successful execution, while a non-zero value indicates a non-success or error status.

Question 4

Write a C program that: • Declares an integer variable • Asks the user to input a value • Displays the entered value back to the user



```
GNU nano 8.7.1
#include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    printf("You entered: %d\n", number);

    return 0;
}
```

Figure 13: C Program

Question 5

What GCC command would you use to: 1) Only preprocess a file without compiling? 2) Generate assembly code but not assemble it? 3) Create an object file without linking?

1) Only preprocess a file?

Answer: `gcc -E file.c`

2) Generate assembly code but not assemble it?

Answer: `gcc -S file.c`

3) Create an object file without linking it?

Answer: `gcc -c file.c`

Question 6

Explain why C is called a "compiled language" and how this differs from interpreted languages.

-C is called a compiled language because its source code is translated into executable machine code before execution, unlike interpreted languages that execute through an interpreter.

VI. Conclusion

This lab provided practical experience with the **fundamentals of C programming and the GCC compilation process in a Linux environment**. Through the practical exercises, basic C programs were created to demonstrate program structure, user input, formatted output, and the use of the `main()` function and return values. These activities helped reinforce the relationship between C source code and program execution.

The lab also provided an understanding of the **four stages of compilation: preprocessing, compilation, assembly, and linking**. By executing each stage separately, the different output files such as `.i`, `.s`, and `.o` were generated and examined. The use of GCC commands also demonstrated how source code is transformed into an executable program.

Furthermore, the practical tasks demonstrated how **return values and exit statuses** can be used to communicate the result of program execution to the operating system. The `echo $?` command was used to verify different return values, including `0` and `1`. Also, the knowledge test question helps to understand basic knowledge and recall important answers taught in class.

Overall, the lab improved practical skills in using the Linux terminal, writing and compiling C programs, understanding the GCC compilation process, and verifying program execution through exit statuses.